Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche, complexité temporelle et structures de données linéaires

Synthèse exécutive : Ce cours pose les bases fondamentales des algorithmes de recherche dans une structure de données linéaire (tableau). L'enjeu central est de comprendre l'impact direct de la structuration et du tri des données sur les performances algorithmiques. Nous analysons la recherche séquentielle (linéaire) de complexité O(n), universelle mais coûteuse sur de grands volumes, et la recherche dichotomique de complexité O(log n), basée sur le paradigme 'diviser pour régner', exigeant un tableau préalablement trié. L'étudiant apprend à manipuler les pointeurs d'indices, évaluer la complexité au pire des cas et effectuer l'arbitrage économique entre le coût du tri et la fréquence des requêtes.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle constitue la méthode la plus intuitive et universelle pour localiser une valeur cible au sein d'une liste ou d'un tableau. L'algorithme initialise un pointeur d'indice à 0 et examine successivement chaque cellule du tableau. À chaque étape, il compare le contenu de la case courante avec la valeur recherchée. Si une égalité est constatée, le traitement s'interrompt immédiatement et l'indice de la cellule est retourné. Si la totalité des n éléments est examinée sans succès, l'algorithme renvoie une valeur conventionnelle signifiant l'absence de l'élément (généralement -1 ou Faux). Sa complexité temporelle dépend directement de la taille n du tableau : au pire des cas (valeur à la fin ou absente), il réalise n comparaisons, soit une complexité notée O(n). Son avantage stratégique réside dans l'absence totale de contrainte sur la structure d'accueil : aucune phase de pré-traitement ou de tri n'est nécessaire.

- Parcours exhaustif et séquentiel du tableau de l'indice 0 jusqu me à l'indice n-1
- Aucune précondition requise sur l'organisation des données (fonctionne sur tableau trié ou non trié)
- Complexité temporelle linéaire O(n) au pire des cas (élément situé en toute fin ou absent)
- Arrêt précoce dès que l'élément cible est identifié (succès)

Cas concret enseignant : *Métaphore du tas de feuilles mortes : Le professeur explique que chercher une clé tombée dans un tas de feuilles désordonné impose de soulever chaque feuille une à une, de la première jusqu'à la dernière, car aucun ordre sous-jacent ne permet de deviner la position de la clé.*

2. La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite l'ordre préexistant des éléments dans un tableau trié pour accélérer de manière spectaculaire la localisation d'une donnée. Au lieu d'un parcours séquentiel, l'algorithme restreint progressivement une zone de recherche délimitée par l'indice 'debut' (initialement 0) et l'indice 'fin' (initialement n-1). À chaque itération, il calcule l'indice central par division entière : milieu = (debut + fin) // 2, puis compare la valeur T[milieu] à la cible : 1) Si T[milieu] == cible, la recherche s'achève avec succès. 2) Si la cible est inférieure à T[milieu], elle ne peut se trouver que dans la première moitié : on déplace le pointeur 'fin' à 'milieu - 1'. 3) Si la cible est supérieure, on déplace le pointeur 'debut' à 'milieu + 1'. Le processus s'itère tant que debut <= fin. En éliminant la moitié des candidats restants à chaque comparaison, la complexité tombe à O(log2 n). Pour un tableau d'un million d'éléments triés, 20 comparaisons suffisent au maximum contre 1 000 000 en recherche séquentielle.

- Précondition absolue et stricte : Le tableau doit obligatoirement être trié au préalable
- Paradigme algorithmique 'Diviser pour Régner' réduisant l'espace de recherche de 50 % à chaque étape
- Gestion dynamique de 3 pointeurs d'indices : debut, fin et milieu = (debut + fin) // 2
- Complexité temporelle logarithmique O(log2 n) au pire des cas

Cas concret enseignant : *Le jeu du 'Devine un nombre entre 1 et 100' : Proposer 1, 2, 3... est une stratégie séquentielle inefficace. Interroger avec 50 élimine immédiatement la moitié des nombres selon que la réponse est 'c'est plus grand' ou 'c'est plus petit'. En poursuivant cette dichotomie, la victoire est garantie en 7 étapes maximum (car 2^7 = 128 > 100).*

3. Arbitrage Algorithmique et Analyse de Performance

En informatique, le choix d'un algorithme dépend du contexte d'utilisation. Bien que la recherche dichotomique surpasse largement la recherche séquentielle, son coût caché réside dans la précondition de tri. Si un tableau n'est pas trié, effectuer un tri rapide ou par fusion exige une complexité de O(n log n). Dès lors, pour effectuer une unique recherche ponctuelle sur un tableau non trié, il est plus efficient d'exécuter une recherche séquentielle en O(n). En revanche, dans le cas d'une base de données ou d'une structure statique interrogée de façon répétée (des milliers de recherches), procéder à un tri initial unique en O(n log n) devient un investissement extrêmement rentable, car chaque recherche ultérieure ne coûtera que O(log n).

- Coût du tri préalable : Trier un tableau non ordonné nécessite au mieux O(n log n)
- Recherche unique : La recherche séquentielle O(n) est plus rapide que de trier puis chercher O(n log n + log n)
- Recherches multiples : Le coût du tri initial O(n log n) est largement amorti si le tableau subit de très nombreuses requêtes O(log n)

Cas concret enseignant : *Analogie du dictionnaire imprimé : La mise en ordre alphabétique des mots exige un travail colossal lors de l'impression initiale (le tri), mais cet effort unique permet ensuite à des millions de lecteurs de trouver un mot en quelques secondes durant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est immensément plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique exige impérativement un tableau trié. Si les données ne sont pas triées au départ, le coût pour les trier (O(n log n)) est supérieur au coût d'une simple recherche séquentielle (O(n)). Si l'on n'a qu'une seule recherche à faire sur un tableau non trié, la recherche séquentielle reste donc plus rapide et économique.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise systématiquement la division entière (notée // en Python ou Integer Division en pseudo-code) : milieu = (debut + fin) // 2. Si la somme est impaire, la tranche décimale est tronquée, ce qui sélectionne automatiquement la case centrale inférieure. L'algorithme reste parfaitement exact et rigoureux.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] composé de n = 10 éléments (indices 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur cible 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : - Calcul du milieu : milieu = (0 + 9) // 2 = 4. - Valeur examinée : T[4] = 19. - Comparaison : Cible 42 > 19. La valeur recherchée est strictly supérieure, elle se trouve donc dans la sous-partie droite. - Mise à jour du pointeur debut : debut = milieu + 1 = 5. Étape 2 : - Nouvelle zone de recherche : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) // 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : T[7] == 42. La cible est identifiée ! - Succès de la recherche.

Résultat : La valeur 42 a été localisée à l'indice 7 du tableau en seulement 2 comparaisons grâce à la dichotomie, alors qu'une recherche séquentielle aurait nécessité 8 comparaisons (de T[0]=3 jusqu'à T[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la précondition indispensable et obligatoire pour pouvoir exécuter une recherche dichotomique sur un tableau ?

R : Le tableau doit être strictement trié au préalable (par ordre croissant ou décroissant).

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle effectuée sur un tableau de n éléments ?

R : La complexité temporelle est linéaire, notée O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : Elle nécessite au maximum 11 comparaisons au pire des cas (puisque log2(1024) = 10 itérations de division par 2).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765